

Implementation of a Graph-Based LLM Workflow for Hardware Verification

Reimplementing AutoBench with LangGraph and Extending it with VCD Waveform Analysis

Yash Jaiswal
71535

Research Project Exposé

Motivation

Hardware functional verification is one of the most time-intensive stages of digital circuit design. According to the 2022 Wilson Research Group study, engineers spend approximately 49% of their working time on verification alone [1]. The standard tool for this is the testbench, a simulation harness that feeds stimuli into a Design Under Test (DUT) and checks its outputs. Writing effective testbenches is slow and error-prone, motivating the use of Large Language Models (LLMs) to automate the process.

AutoBench (MLCAD 2024) introduced the first systematic LLM-based testbench generator, using a multi-stage pipeline with a hybrid Verilog driver and Python checker, achieving a 57% improvement in the Eval2 pass metric over direct LLM generation, and a 3.36x improvement for sequential circuits [2]. CorrectBench (DATE 2025) extended this with self-correction, reaching a 70.13% pass ratio [3]. However, both systems share two key limitations: they are implemented as rigid, monolithic Python scripts with hardcoded control flow, and they evaluate results as binary pass/fail signals, discarding the rich waveform data produced during simulation.

This project addresses both gaps. The base contribution is a LangGraph reimplementation of the AutoBench pipeline as a structured graph with defined nodes, typed state, and conditional routing [4]. The novel contribution is a VCD waveform analysis module: after simulation, the system captures the Value Change Dump file, parses it into a structured LLM-readable signal table, and uses the LLM to localise faults beyond simple pass/fail, mirroring how a human engineer reads a waveform viewer. Experiments use the VerilogEval-Human benchmark of 156 problems [5], enabling direct comparison with prior work.

Work Packages

1. Literature Review and Environment Setup

Study AutoBench and CorrectBench papers and source code; complete the LangGraph tutorial; understand the VCD format specification [6]; set up Icarus Verilog, Python environment, and free LLM API credentials (Gemini / Groq / Qwen). Define the Eval0/Eval1/Eval2 evaluation criteria to be used throughout.

- Literature summary and research gap statement
- Operational development environment (Python, Icarus Verilog, LLM API)

2. LangGraph Reimplementation

Restructure the AutoBench pipeline as a proper LangGraph state graph. Each stage (spec generation, scenario planning, driver/checker generation, simulation, debug, reboot) becomes a stand-alone node with a typed interface. Conditional edges replace hardcoded if/else logic. Run on all 156 VerilogEval problems to establish a baseline.

- LangGraph graph with defined nodes and conditional routing
- Baseline Eval0/Eval1/Eval2 pass@1/5/10 results on 156-problem dataset

3. VCD Waveform Parsing and LLM Analysis

Extracting the VCD files from the verilog-eval tests and having the LLM evaluate the waveforms directly. Implement a VCDParserNode that resolves signal identifiers, reconstructs time-series state, and renders a structured input/output table. A WaveformAnalysisNode then prompts the LLM with this table to produce a verdict and diagnosis, which feeds back into the graph for targeted repair.

4. Evaluation and Reporting

Run a full evaluation on benchmark problems. Produce comparison tables, coverage distribution plots, and ablation results. Write the final project report and prepare the reproducible code repository.

- Full Eval0/Eval1/Eval2 results table vs. prior work
- Final written report and presentation slides
- Public GitHub repository with code, dataset, and results
- Defined evaluation criteria aligned with AutoBench's AutoEval framework

Tentative Time Schedule

Period	Tasks	WP
11.05 - 24.05	Study papers & code. Complete LangGraph tutorial. Set up the environment.	WP 1
25.05 - 07.06	Reproduce AutoBench on sample problems. Design LangGraph node interfaces.	WP 1→2
08.06 - 21.06	Implement a full LangGraph graph. Run baseline on all 156 problems.	WP 2
22.06 - 5.07	Implement VCDParserNode. Unit-test on diverse sample circuits.	WP 3
06.07 - 19.07	Implement WaveformAnalysisNode. Integrate repair loop. Run ablation.	WP 3
20.07 - 26.07	Full benchmark evaluation. Generate comparison tables and plots.	WP 4
27.07 - 09.08	Write a report, prepare slides, finalise, and submit to the repository.	WP 4

References

- [1] H. Foster, "The 2022 Wilson Research Group Functional Verification Study," Siemens Verification Horizons Blog, Dec. 2022. [Online]. Available: <https://blogs.sw.siemens.com/verificationhorizons/>
- [2] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "AutoBench: Automatic Testbench Generation and Evaluation Using LLMs for HDL Design," in Proc. ACM/IEEE Int. Symp. Machine Learning for CAD (MLCAD), Salt Lake City, UT, USA, Sep. 2024. DOI: 10.1145/3670474.3685956
- [3] R. Qiu et al., "CorrectBench: Automatic Testbench Generation with Functional Self-Correction using LLMs for HDL Design," in Proc. Design, Automation and Test in Europe (DATE), 2025. arXiv:2411.08510
- [4] LangChain, Inc., "LangGraph: Build Stateful, Multi-Actor Applications with LLMs," 2024. [Online]. Available: <https://github.com/langchain-ai/langgraph>
- [5] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating Large Language Models for Verilog Code Generation," in Proc. IEEE/ACM Int. Conf. Computer Aided Design (ICCAD), 2023.

[6] ChipVerify, "Verilog VCD (Value Change Dump)," [Online]. Available: <https://www.chipverify.com/verilog/verilog-vcd>